Padrões de Projeto

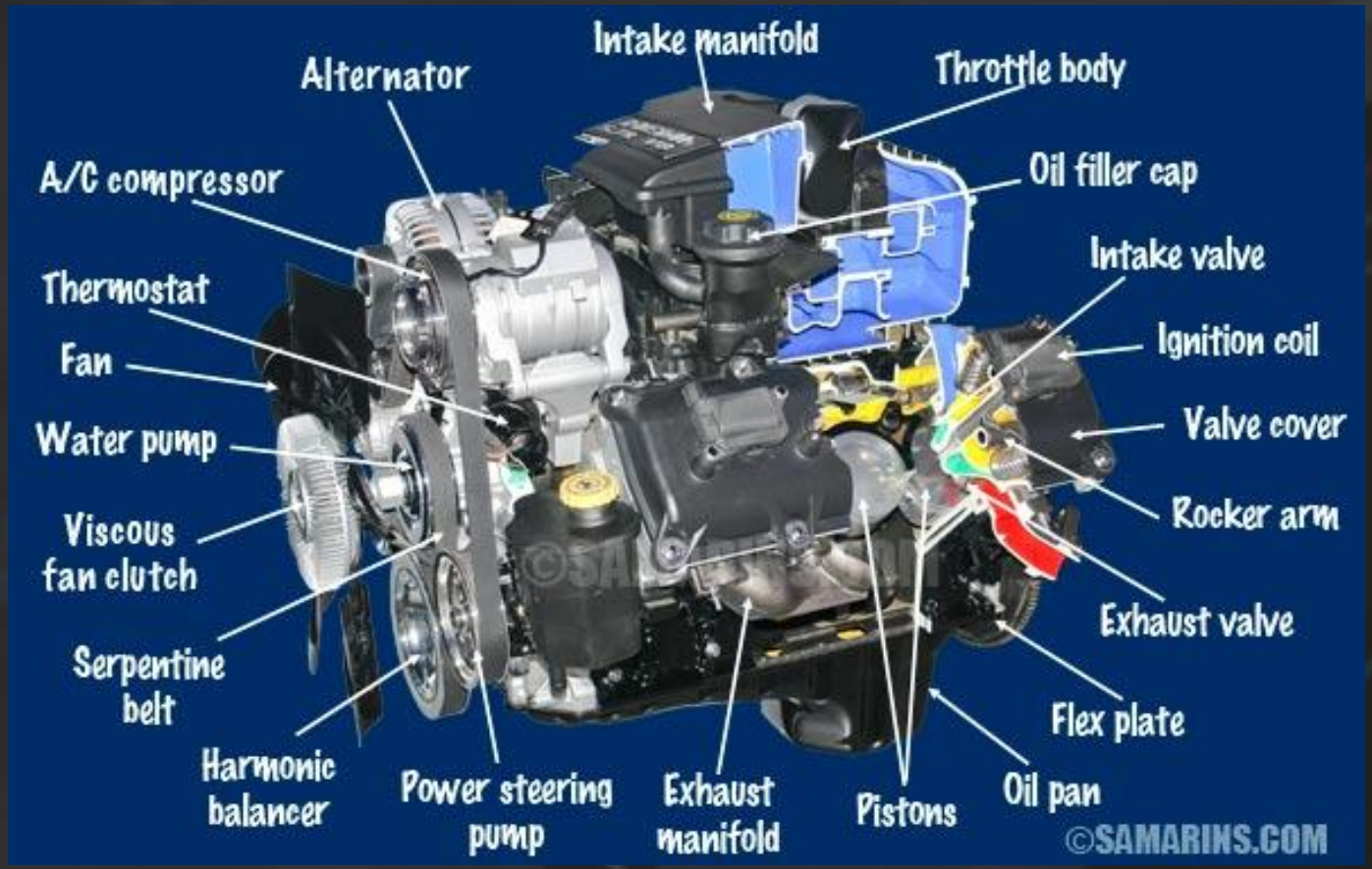
FURB
UNIVERSIDADE DE BLUMENAU

Utilidade #1: Reúso de projeto

- Suponha que eu tenho um problema de projeto
- Pode existir um padrão que resolve esse problema
- Vantagem: ganho tempo e não preciso reinventar a roda

Utilidade #2: Vocabulário para comunicação

- Vocabulário para discussões, documentação, etc


Intake manifold
Alternator
Throttle body
A/C compressor
Oil filler cap
Intake valve
Thermostat
Ignition coil
Fan
Valve cover
Water pump
Rocker arm
Viscous fan clutch
Exhaust valve
Serpentine belt
Flex plate
Harmonic balancer
Power steering pump
Exhaust manifold
Pistons
Oil pan
©SAMARINS.COM

Quais são as "peças" que posso usar no design do meu software?

FURB
UNIVERSIDADE DE BLUMENAU

Criacionais	Estruturais	Comportamentais
Abstract Factory	Proxy	Strategy
Factory Method	Adapter	Observer
Singleton	Facade	Template Method
Builder	Decorator	Visitor
Prototype	Bridge	Chain of Responsibility
	Composite	Command
	Flyweight	Interpreter
		Iterator
		Mediator
		Memento
		State

23 padrões

Padrões de Projeto

- Soluções recorrentes para problemas de projeto enfrentados por engenheiros de software

Estrutura da Apresentação dos Padrões

- Contexto (sistema ou parte de um sistema)
- Problema (de projeto)
- Solução (por meio de um padrão de projeto)

Importante

- Padrões de projeto ajudam em design for change
- Facilitam mudanças futuras no código
- Se o código dificilmente vai precisar de mudança, então uso de padrões é um exemplo de overengineering

Complementando a comparação anterior:

Quais são as "peças" que posso usar no design do meu software para torná-lo flexível e fácil de modificar?

Código no GitHub

https://github.com/afburger/FURB-2026_01-PS_II-115_02_001/tree/main/Padroes_Projeto

(1) Fábrica

Relembrando a estrutura da explicação

- Contexto
- Problema
- Solução

Contexto: Sistema que usa canais de comunicação

```
void f() {
  TCPChannel c = new TCPChannel();
  ...
}

void g() {
  TCPChannel c = new TCPChannel();
  ...
}

void h() {
  TCPChannel c = new TCPChannel();
  ...
}
```

Problema

- Alguns clientes vão precisar de usar UDP, em vez de TCP
- Como parametrizar as chamadas de **new**?
- Estão espalhadas em vários pontos do código
- Como criar um projeto que facilite essa mudança?

Solução: **Padrão Fábrica**

- Fábrica: método que centraliza a criação de certos objetos

```
class ChannelFactory {
   public static Channel create() { // método fábrica estático
     return new TCPChannel();
   }
}
```

Único ponto de mudança se quisermos mudar para UDP

Sem Fábrica

```
void f() {
  TCPChannel c = new TCPChannel();
  ...
}

void g() {
  TCPChannel c = new TCPChannel();
  ...
}

void h() {
  TCPChannel c = new TCPChannel();
  ...
}
```

FURB
UNIVERSIDADE DE BLUMENAU

Sem Fábrica

```
void f() {
  TCPChannel c = new TCPChannel();
  ...
}

void g() {
  TCPChannel c = new TCPChannel();
  ...
}

void h() {
  TCPChannel c = new TCPChannel();
  ...
}
```

⇒

Com Método Fábrica Estático

```
void f() {
  Channel c = ChannelFactory.create();
  ...
}

void g() {
  Channel c = ChannelFactory.create();
  ...
}

void h() {
  Channel c = ChannelFactory.create();
  ...
}
```

FURB
UNIVERSIDADE DE BLUMENAU

(2) Singleton

```java
void f() {
  Logger log = new Logger();
  log.println("Executando f");
  ...
}
void g() {
  Logger log = new Logger();
  log.println("Executando g");
  ...
}
void h() {
  Logger log = new Logger();
  log.println("Executando h");
  ...
}
```

Contexto: Classe Logger

FURB
UNIVERSIDADE DE BLUMENAU

```java
void f() {
  Logger log = new Logger();
  log.println("Executando f");
  ...
}
void g() {
  Logger log = new Logger();
  log.println("Executando g");
  ...
}
void h() {
  Logger log = new Logger();
  log.println("Executando h");
  ...
}
```

Problema:

Cada método cliente usa sua própria instância de Logger

Problema

- Como fazer com que todos os clientes usem a mesma instância de Logger?
- Na verdade, deve existir uma única instância de Logger
- Todo "log" seria registrado no mesmo arquivo

Solução: **Padrão Singleton**

- Transformar a classe Logger em um Singleton
- Singleton: classe que possui no máximo uma instância

```java
class Logger {

  private Logger() {} // proíbe clientes chamar new Logger()

  private static Logger instance; // instância única da classe

  public static Logger getInstance() {
    if (instance == null) // 1a vez que chama-se getInstance
      instance = new Logger();
    return instance;
  }

  public void println(String msg) {
    // registra msg na console, mas poderia ser em arquivo
    System.out.println(msg);
  }
}
```

```java
class Logger {

  private Logger() {} // proíbe clientes chamar new Logger()

  private static Logger instance; // instância única da classe

  public static Logger getInstance() {
    if (instance == null) // 1a vez que chama-se getInstance
      instance = new Logger();
    return instance;
  }

  public void println(String msg) {
    // registra msg na console, mas poderia ser em arquivo
    System.out.println(msg);
  }
}
```

```java
class Logger {

  private Logger() {} // proíbe clientes chamar new Logger()

  private static Logger instance; // instância única da classe

  public static Logger getInstance() {
    if (instance == null) // 1a vez que chama-se getInstance
      instance = new Logger();
    return instance;
  }

  public void println(String msg) {
    // registra msg na console, mas poderia ser em arquivo
    System.out.println(msg);
  }
}
```

1

2

FURB
UNIVERSIDADE DE BLUMENAU

```java
class Logger {

  private Logger() {} // proíbe clientes chamar new Logger()

  private static Logger instance; // instância única da classe

  public static Logger getInstance() {
    if (instance == null) // 1a vez que chama-se getInstance
      instance = new Logger();
    return instance;
  }

  public void println(String msg) {
    // registra msg na console, mas poderia ser em arquivo
    System.out.println(msg);
  }
}
```

```java
void g() {
  Logger log = Logger.getInstance();
  log.println("Executando g");
  ...
}

void h() {
  Logger log = Logger.getInstance();
  log.println("Executando h");
  ...
}
```

Mesma instância

FURB
UNIVERSIDADE DE BLUMENAU

(3) Proxy

Contexto: Função que pesquisa livros

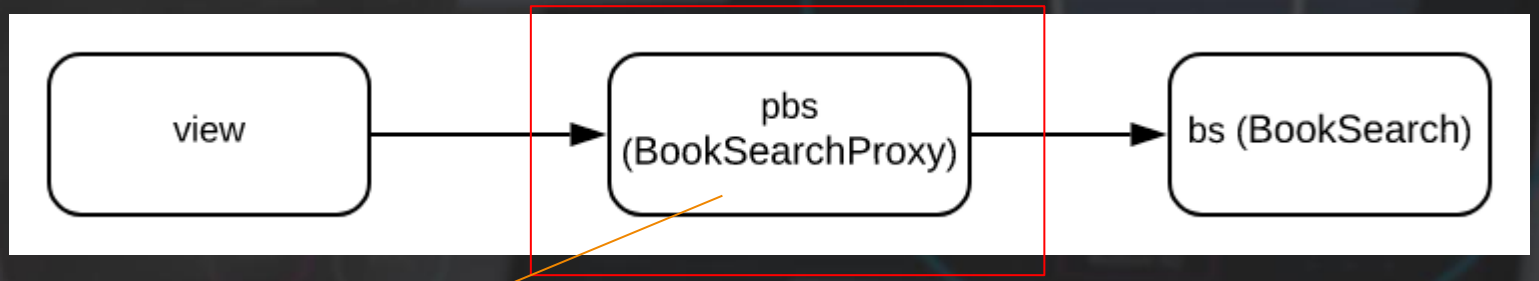
```
class BookSearch {
   ...
   Book getBook(String ISBN) { ... }
   ...
}
```

Problema: Inserir um cache (para melhorar desempenho)

- Se "livro no cache"
  - retorna livro imediatamente
  - caso contrário, continua com a pesquisa
- Porém, não queremos modificar o código de BookSearch
  - Classe já está funcionando
  - Já tem um desenvolvedor acostumado a mantê-la, etc

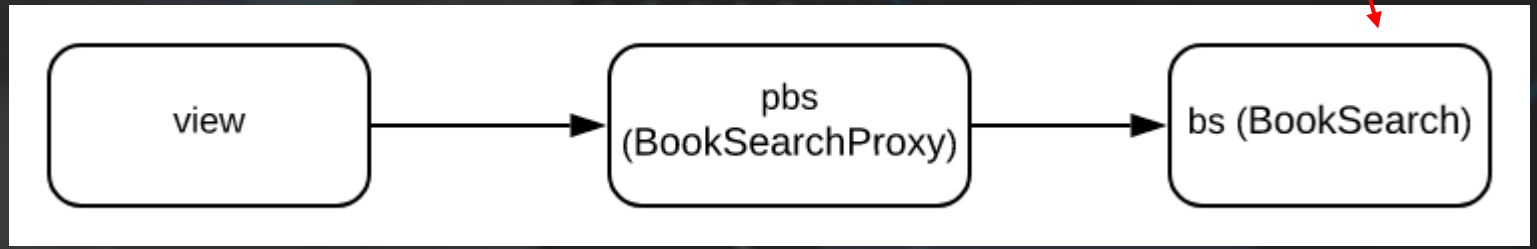
Solução: **Padrão Proxy**

- Proxy: objeto intermediário entre cliente e um objeto base
- Clientes não "falam" mais com o objeto base (diretamente)
- Eles têm que passar antes pelo proxy

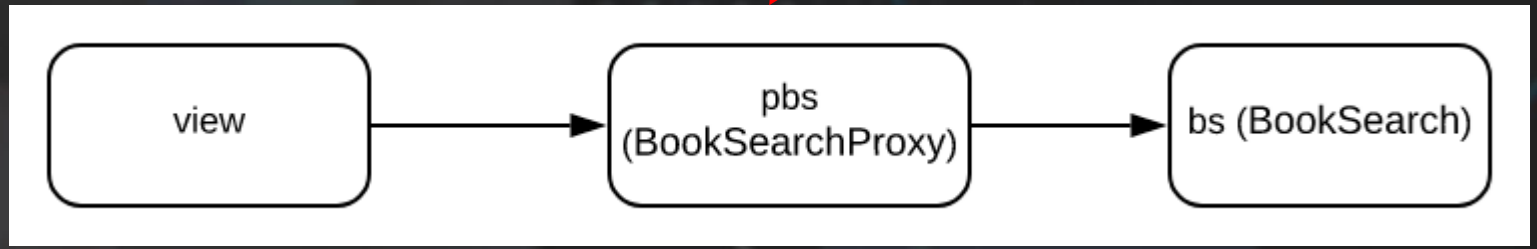


Implementa a lógica do cache

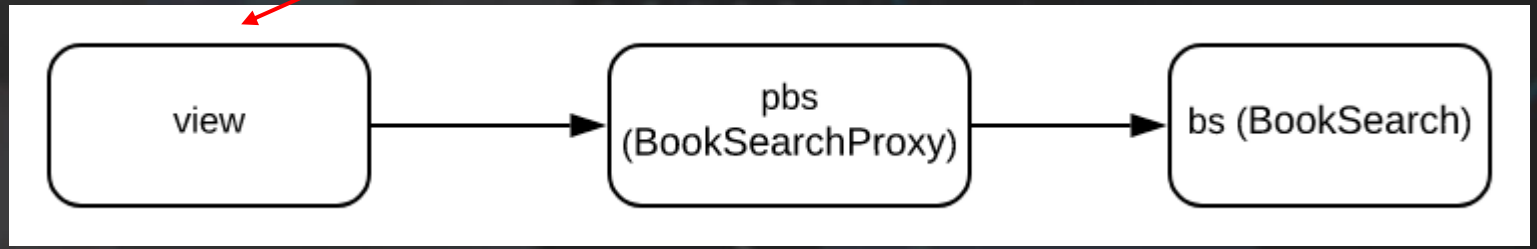
```
void main() {
  BookSearch bs = new BookSearch();
  BookSearchProxy pbs;
  pbs = new BookSearchProxy(bs);
  ...
  View view = new View(pbs);
  ...
}
```

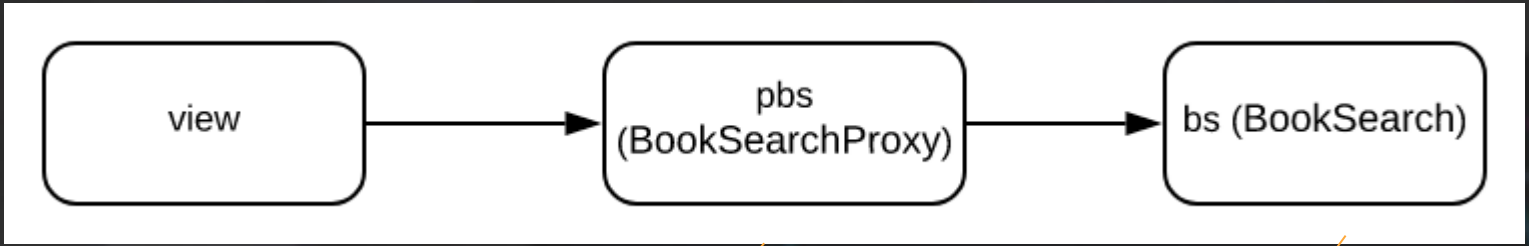


```
void main() {
  BookSearch bs = new BookSearch();
  BookSearchProxy pbs;
  pbs = new BookSearchProxy(bs);
  ...
  View view = new View(pbs);
  ...
}
```



```
void main() {
  BookSearch bs = new BookSearch();
  BookSearchProxy pbs;
  pbs = new BookSearchProxy(bs);
  ...
  View view = new View(pbs);
  ...
}
```





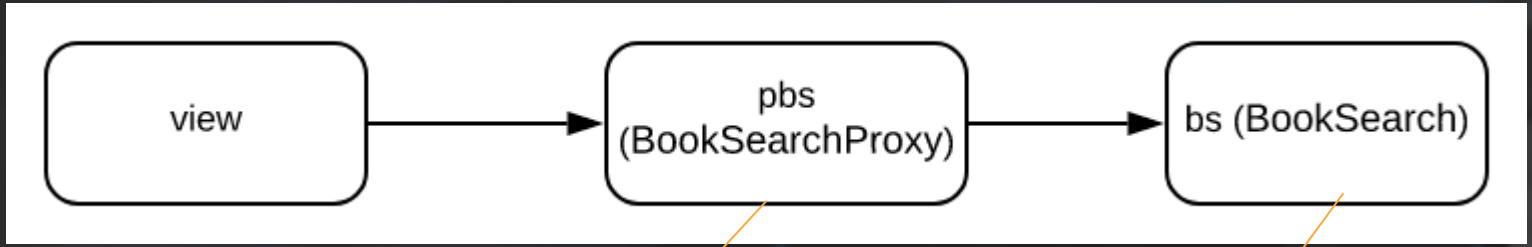
Requisito Não-Funcional

Requisito Funcional

Logo, um proxy:

(1) ajuda a melhorar qual propriedade de projeto?

(2) é um aplicação de qual princípio SOLID?



Requisito Não-Funcional

Requisito Funcional

Logo, um proxy:

(1) ajuda a melhorar qual propriedade de projeto?

Propriedade de projeto: Baixo acoplamento / separação de responsabilidades

(2) é um aplicação de qual princípio SOLID?

Princípio SOLID principal: Responsabilidade Única o objeto real tem apenas as lógicas de negócio e o proxy, assume as demais reponsabilidades.

Princípio SOLID secundário: Open / Close: adiciona comportamentos sem mudar a classe original.

O código de BookSearchProxy está no :

https://github.com/afburger/FURB-2026_01-PS_II-115_02_001/blob/main/Padroes_Projeto/3-proxy/Main.java

(4) Adaptador

Contexto: Sistema para Controlar Projetores Multimídia

```
class ProjetorSamsung {
  public void turnOn() { ... }
  ...
}

class ProjetorLG {
  public void enable(int timer) { ... }
  ...
}
```

Classes fornecidas pelos fabricantes dos projetores

Problema

- No sistema de controle de multimídias, eu gostaria de manipular uma única interface Projetor

```
interface Projetor {

  void liga();

}
...
class SistemaControleProjetores {

  void init(Projetor projetor) {
    projetor.liga();  // liga qualquer projetor
  }

}
```

Sendo mais técnico, eu gostaria de aproveitar e fazer uso de qual princípio SOLID?

FURB
UNIVERSIDADE DE BLUMENAU

Sendo mais técnico, eu gostaria de aproveitar e fazer uso de qual princípio SOLID?

Princípio da Inversão de Dependência

Problema (cont.)

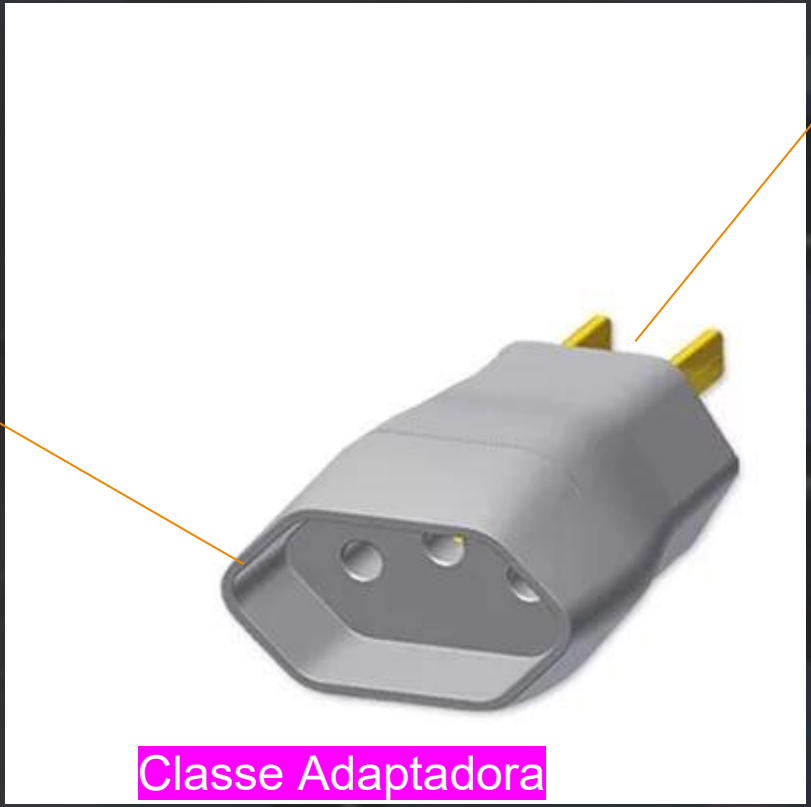
```
class ProjetorSamsung {
  public void turnOn() { ... }
  ...
}

class ProjetorLG {
  public void enable(int timer) { ... }
  ...
}
```

- São classes dos fabricantes dos projetores
- Eu não tenho acesso a elas. Para, por exemplo, fazer com que elas implementem a interface Projetor

Solução: **Padrão Adaptador**

Padrão (ou interface) de saída

Padrão (ou interface) de entrada



Solução: Implementar uma Classe Adaptadora

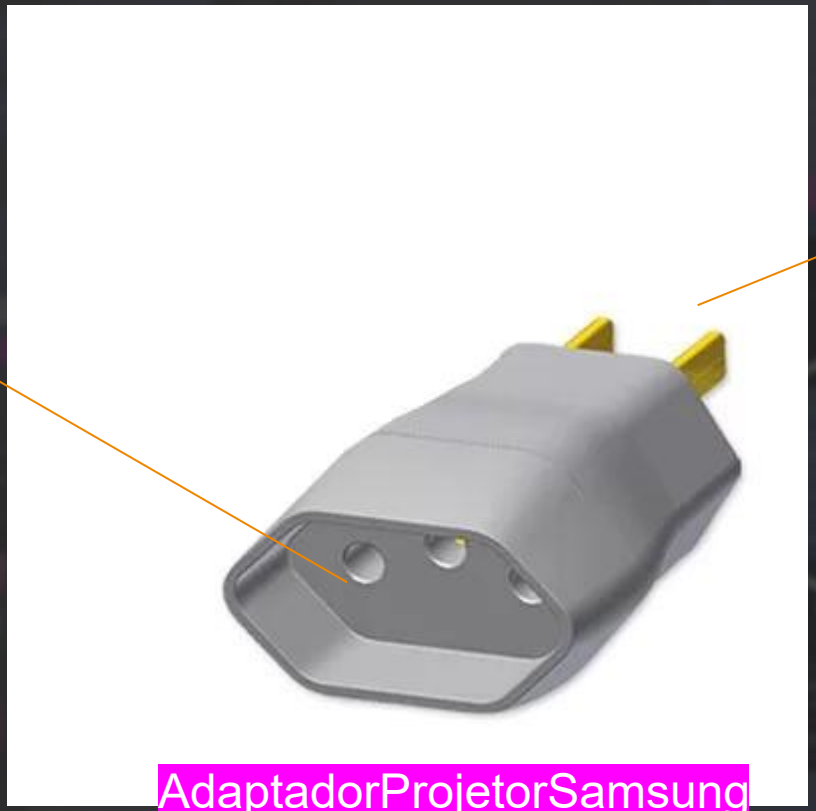
```
class AdaptadorProjetorSamsung implements Projetor {

    private ProjetorSamsung projetor;

    AdaptadorProjetorSamsung (ProjetorSamung projetor) {
      this.projetor = projetor;
    }

    public void liga() {
      projetor.turnOn();
    }

}
```

Solução: Implementar uma Classe Adaptadora

```
class AdaptadorProjetorSamsung implements Projetor {

    private ProjetorSamsung projetor;

    AdaptadorProjetorSamsung (ProjetorSamung projetor) {
      this.projetor = projetor;
    }

    public void liga() {
      projetor.turnOn();
    }

}
```

ProjetorSamsung

Projetor



AdaptadorProjetorSamsung

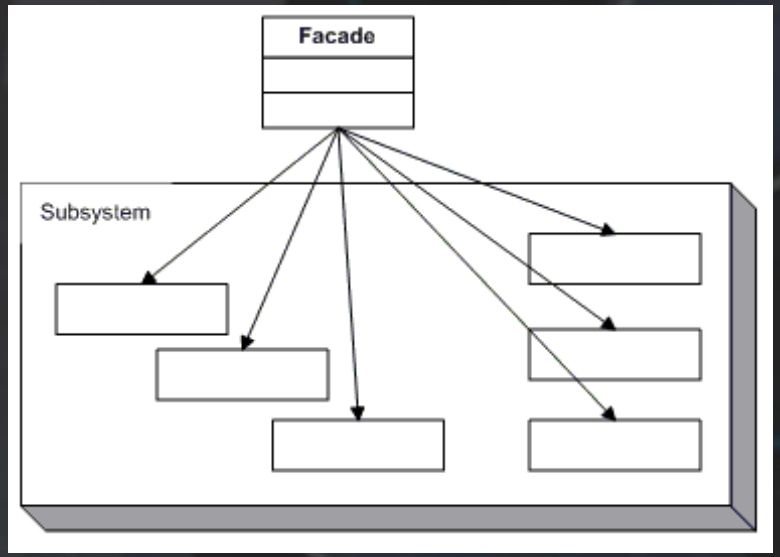
(5) Fachada

Contexto & Problema

- Contexto: módulo M usado por diversos outros módulos
- Problema:
  - Interface de M é complexa
  - Clientes reclamam que é difícil usar a interface de M

Solução

- Criar uma interface mais simples para M, chamada de **Fachada**.



https://refactoring.guru/design-patterns/facade

Exemplo: Interpretador

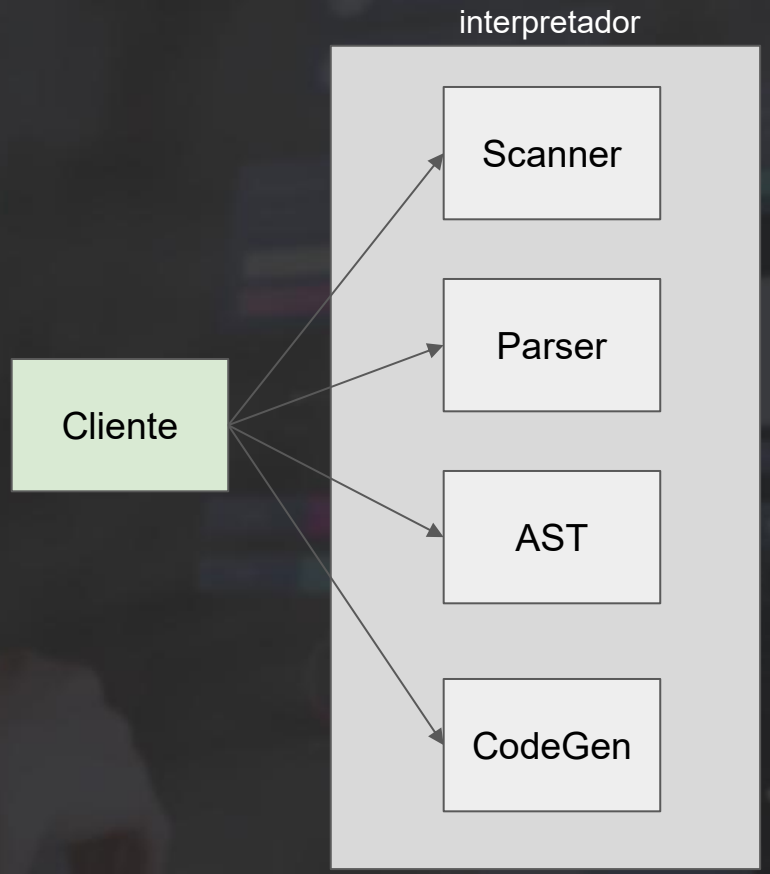
```
Scanner s = new Scanner("prog1.x");
Parser p = new Parser(s);
AST ast = p.parse();
CodeGenerator code = new CodeGenerator(ast);
code.eval();
```

```
Scanner s = new Scanner("prog1.x");
Parser p = new Parser(s);
AST ast = p.parse();
CodeGenerator code = new CodeGenerator(ast);
code.eval();
```

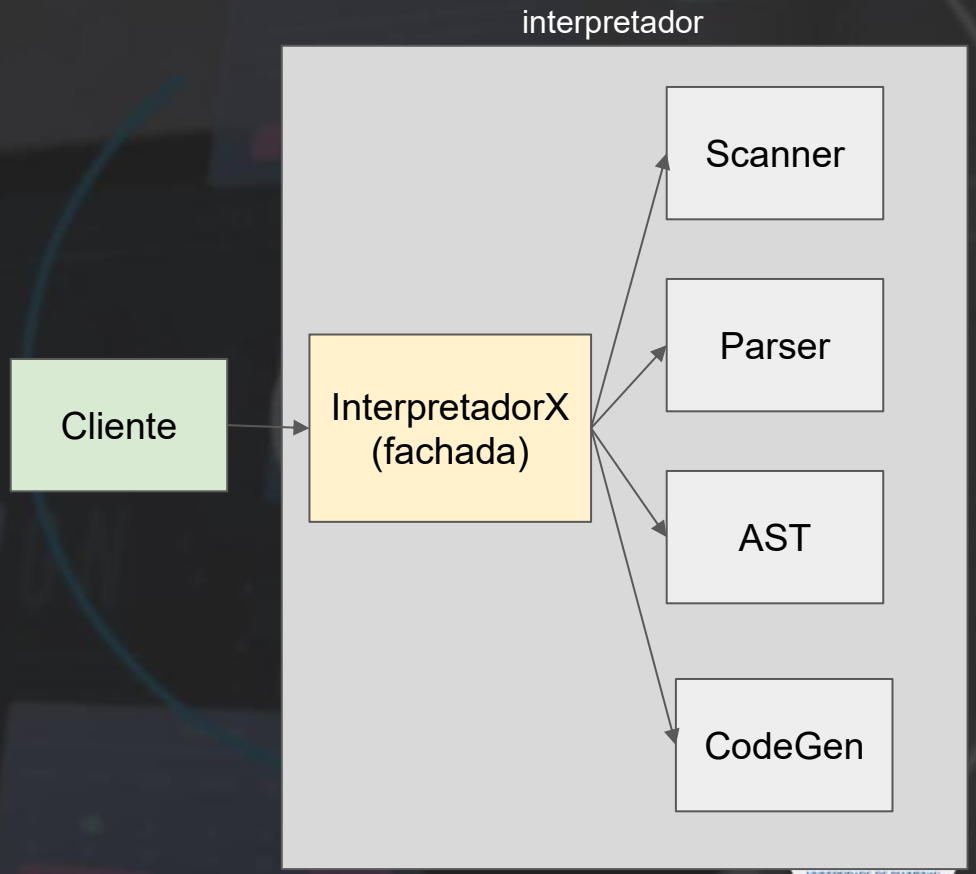
```
new InterpretadorX("prog1.x").eval();
```

Interface mais simples,
que a de cima

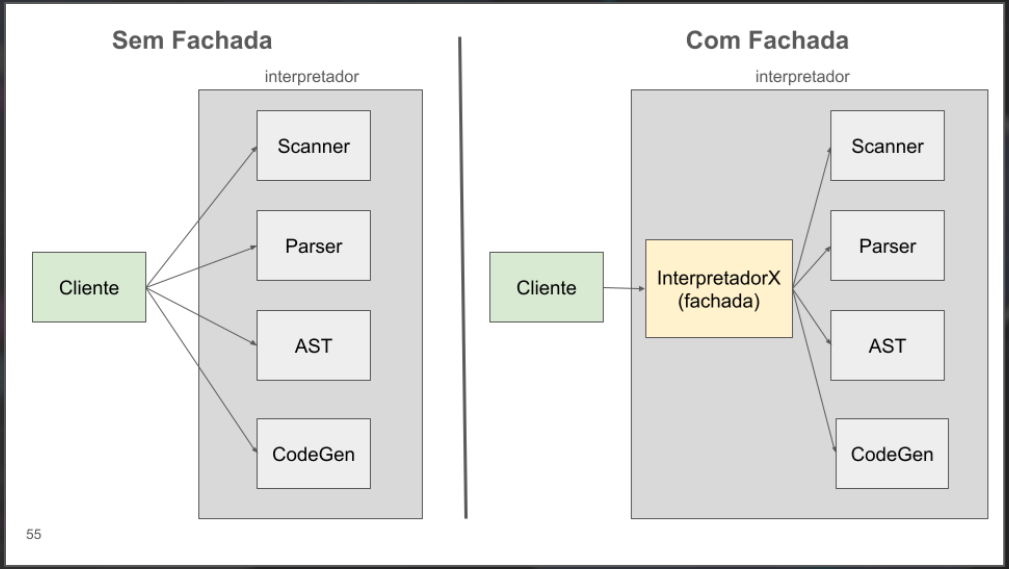
Sem Fachada



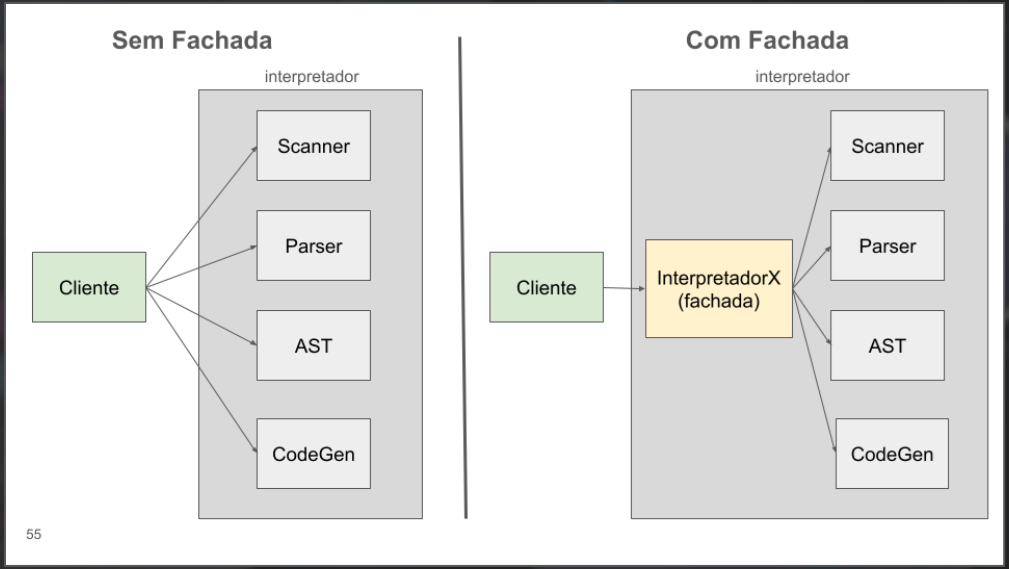
Com Fachada



Na figura anterior, considerando o cliente, qual propriedade do seu projeto estamos melhorando?

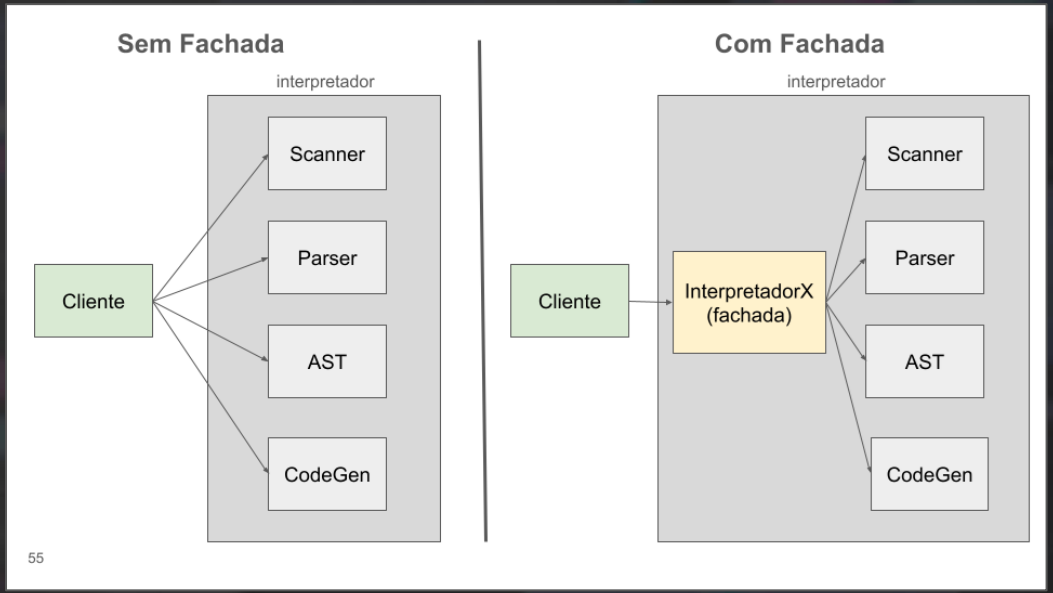


Na figura anterior, considerando o cliente, qual propriedade do seu projeto estamos melhorando?

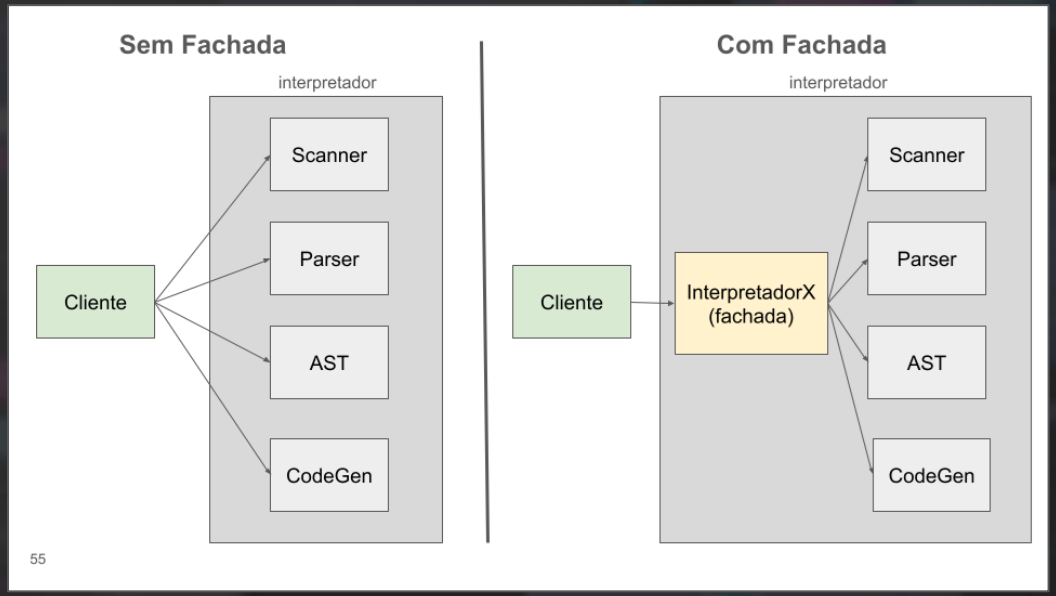


Baixo acoplamento e Ocultamento de informação

Por outro lado, se **não** tomarmos cuidado, qual princípio de projeto pode ser violado quando se usa uma fachada?



Por outro lado, se **não** tomarmos cuidado, qual princípio de projeto pode ser violado quando se usa uma fachada?



Responsabilidade ùnica

(6) Decorador

FURB
UNIVERSIDADE DE BLUMENAU

Contexto: Sistema que usa canais de comunicação

(mesmo usado para o padrão Fábrica)

```
interface Channel {
    void send(String msg);
    String receive();
}

class TCPChannel implements Channel {
    ...
}
class UDPChannel implements Channel {
    ...
}
```

Problema: Precisamos adicionar funcionalidades extras em canais

- Os canais default (TCP, UDP) não são suficientes
- Precisamos também de canais com:
  - Compactação e descompactação
  - Buffers (ou caches)
  - Logging
  - etc

Solução: **Padrão Decorador**

- Resolve esse problema - adicionar funcionalidades em uma classe - por meio de composição e sem gerar um número excessivo de classes

Mais um exemplo

```
channel = new BufferChannel(new ZipChannel (new TCPChannel()));
// TCPChannel com compactação e um buffer associado
```

Mais um exemplo

```
channel = new BufferChannel(new ZipChannel (new TCPChannel()));
// TCPChannel com compactação e um buffer associado
```

Mais um exemplo

```
channel = new BufferChannel(new ZipChannel (new TCPChannel()));
// TCPChannel com compactação e um buffer associado
```

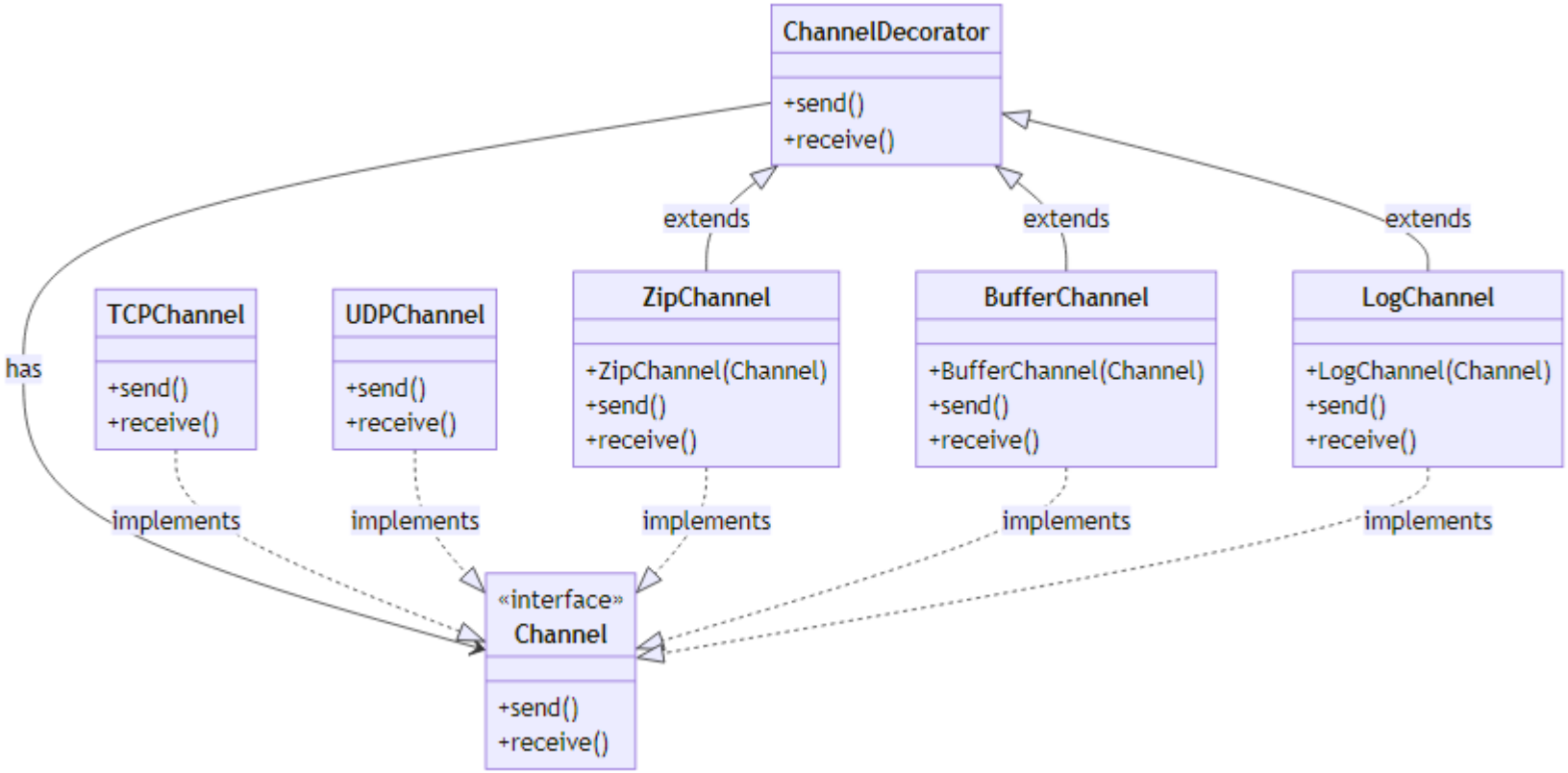
Comparação

```
channel = new BufferChannel(new ZipChannel (new TCPChannel()));
// TCPChannel com compactação e um buffer associado
```

Dentro de uma caixa, tem outra caixa, que tem outra caixa … até chegar no presente. Isto é, até chegar em TCPChannel ou UDPChannel

Implementação do Padrão Decorador

FURB
UNIVERSIDADE DE BLUMENAU


ChannelDecorator
+send()
+receive()
extends
extends
extends
has
TCPChannel
+send()
+receive()
UDPChannel
+send()
+receive()
ZipChannel
+ZipChannel(Channel)
+send()
+receive()
BufferChannel
+BufferChannel(Channel)
+send()
+receive()
LogChannel
+LogChannel(Channel)
+send()
+receive()
implements
implements
implements
implements
implements
«interface»
Channel
+send()
+receive()

ChannelDecorator

```
class ChannelDecorator implements Channel {

  protected Channel channel;

  public ChannelDecorator(Channel channel) {
    this.channel = channel;
  }

  public void send(String msg) {
    channel.send(msg);
  }

  public String receive() {
    return channel.receive();
  }

}
```

Todos os decoradores são subclasses dessa classe única

Implementação: ZipChannel

ZipChannel

```
class ZipChannel extends ChannelDecorator {

    public ZipChannel(Channel c) {
     super(c);
    }

    public void send(String msg) {
     "compacta mensagem msg"
     super.channel.send(msg);
    }

    public String receive() {
     String msg = super.channel.receive();
     "descompacta mensagem msg"
     return msg;
    }

}
```

Exemplo

```
channel = new BufferChannel(new ZipChannel (new TCPChannel()));
// TCPChannel com compactação e um buffer associado
```

```
class BufferChannel extends ChannelDecorator {
    ... super.channel
}
```

Exemplo

```
channel = new BufferChannel(new ZipChannel (new TCPChannel()));
// TCPChannel com compactação e um buffer associado
```

```
class ZipChannel extends ChannelDecorator {
    ... super.channel
}
```

Exemplo

```
channel = new BufferChannel(new ZipChannel (new TCPChannel()));
// TCPChannel com compactação e um buffer associado
```

```
class TCPChannel implements Channel {
    // canal "final"
}
```

(7) Strategy

Contexto: Biblioteca de Estruturas de Dados

```
class MyList {

  ... // dados de uma lista
  ... // métodos de uma lista: add, delete, search

  public void sort() {
    ... // ordena a lista usando Quicksort
  }

}
```

Problema

- Classe MyList **não** está aberta a extensões
- Possível extensão: mudar o algoritmo de ordenação
- Usar ShellSort, HeapSort, etc

Solução: **Padrão Strategy**

- Objetivo:
  - Parametrizar os algoritmos usados por uma classe
  - Tornar uma classe "aberta" a novos algoritmos
- No exemplo: novos algoritmos de ordenação

Passo #1: Criar uma hierarquia de "estratégias" (estratégia = algoritmo)

SortStrategy poderia ser também uma interface

```
abstract class SortStrategy {
  abstract void sort(MyList list);
}
class QuickSortStrategy extends SortStrategy {
  void sort(MyList list) { ... }
}
class ShellSortStrategy extends SortStrategy {
  void sort(MyList list) { ... }
}
```

FURB
UNIVERSIDADE DE BLUMENAU

Passo #2: Modificar MyList para usar a hierarquia de estratégias

FURB
UNIVERSIDADE DE BLUMENAU

```java
class MyList {

  ... // dados de uma lista
  ... // métodos de uma lista: add, delete, search

  private SortStrategy strategy;

  public MyList() {
    strategy = new QuickSortStrategy();
  }

  public void setSortStrategy(SortStrategy strategy) {
    this.strategy = strategy;
  }

  public void sort() {
    strategy.sort(this);
  }

}
```

```java
class MyList {

  ... // dados de uma lista
  ... // métodos de uma lista: add, delete, search

  private SortStrategy strategy;

  public MyList() {
    strategy = new QuickSortStrategy();
  }

  public void setSortStrategy(SortStrategy strategy) {
    this.strategy = strategy;
  }

  public void sort() {
    strategy.sort(this);
  }

}
```

```java
class MyList {

  ... // dados de uma lista
  ... // métodos de uma lista: add, delete, search

  private SortStrategy strategy;

  public MyList() {
    strategy = new QuickSortStrategy();
  }

  public void setSortStrategy(SortStrategy strategy) {
    this.strategy = strategy;
  }

  public void sort() {
    strategy.sort(this);
  }

}
```

```java
class MyList {

  ... // dados de uma lista
  ... // métodos de uma lista: add, delete, search

  private SortStrategy strategy;

  public MyList() {
    strategy = new QuickSortStrategy();
  }

  public void setSortStrategy(SortStrategy strategy) {
    this.strategy = strategy;
  }

  public void sort() {
    strategy.sort(this);
  }

}
```

(8) Observador

Contexto: Sistema de uma Estação Meteorológica

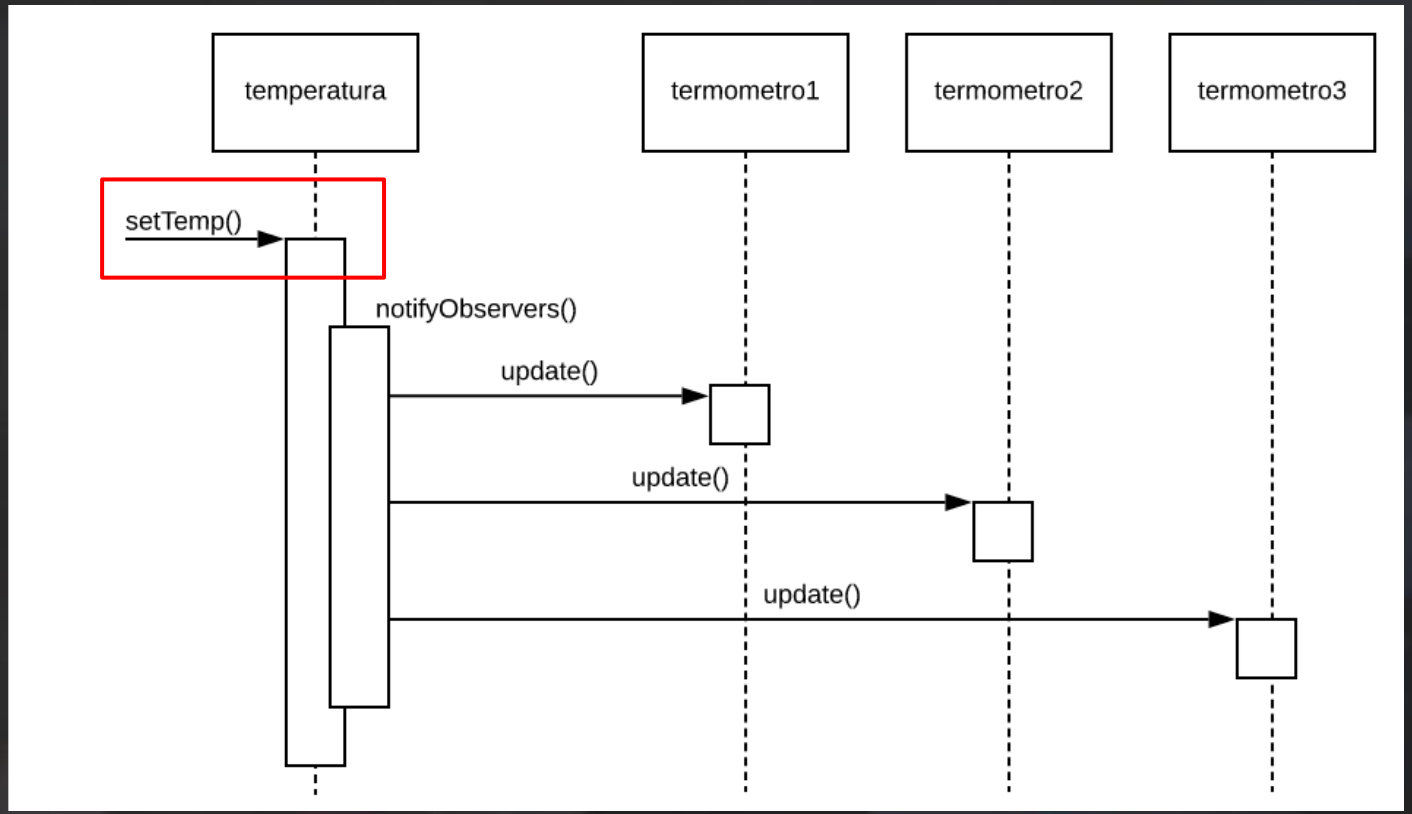
- Duas classes principais:
  - Temperatura
  - Termômetro
- Diversos termômetros: digital, analógico, web, celular, etc
- Se a temperatura mudar, os termômetros devem ser atualizados

Problema

- Não queremos acoplar Temperatura a Termômetros
- Motivos:
  - Tornar classe de dados (modelo) independente de classes de visão (interface com usuários)
  - Tornar flexível a adição de um novo tipo de termômetro no sistema

Solução: **Padrão Observador**

- Implementa uma relação do tipo um-para-muitos entre os seguintes objetos
  - Sujeito (Temperatura)
  - Observadores (Termômetros)
- Quando o estado de um Sujeito muda, seus Observadores são notificados.
- Mas Sujeito não conhece o tipo concreto de seus Observadores


temperatura
termometro1
termometro2
termometro3
setTemp()
notifyObservers()
update()
update()
update()

Programa Principal

```
void main() {
   Temperatura t = new Temperatura();
   t.addObserver(new TermometroCelsius());
   t.addObserver(new TermometroFahrenheit());
   t.setTemp(100.0);
}
```

Sujeito

Programa Principal

```
void main() {
   Temperatura t = new Temperatura();
   t.addObserver(new TermometroCelsius());
   t.addObserver(new TermometroFahrenheit());
   t.setTemp(100.0);
}
```

Dois observadores

Programa Principal

```
void main() {
   Temperatura t = new Temperatura();
   t.addObserver(new TermometroCelsius());
   t.addObserver(new TermometroFahrenheit());
   t.setTemp(100.0);
}
```

Notifica os termômetros

Classe Temperatura

```
class Temperatura extends Subject {

  private double temp;

  public double getTemp() {
    return temp;
  }

  public void setTemp(double temp) {
    this.temp = temp;
    notifyObservers();
  }

}
```

Classe que implementa addObservers e notifyObservers

Uma classe Termômetro

```
class TermometroCelsius implements Observer{

  public void update(Subject s){
    double temp = ((Temperatura) s).getTemp();
    System.out.println("Temperatura Celsius: " + temp);
  }

}
```

Todos Observadores devem implementar esse método.
Chamada de **notifyObservers** (em Temperatura) resulta na execução de **update** de cada um de seus observadores

Outro Exemplo: Planilhas

- Quando o valor de uma célula X muda, todas as células Y que fazem uso de X são automaticamente atualizadas

	A	B	C
1	**Prova 1**	**Prova 2**	**Total**
2	80	85	165

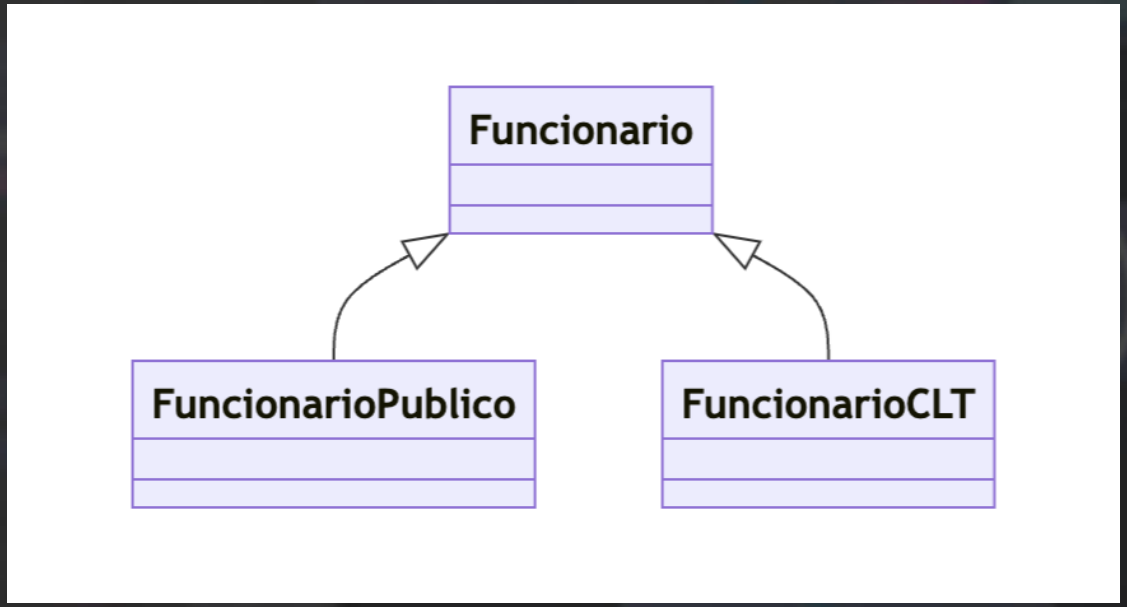
Subject

Observer

```python
class Cell:
    def __init__(self):
        self.value = None
        self.observers = []

    def set_value(self, new_value):
        self.value = new_value
        self.notify_observers()

    def notify_observers(self):
        for observer in self.observers:
            observer.update()

    def add_observer(self, observer):
        self.observers.append(observer)
```

(9) Template Method

Contexto: Folha de Pagamento



Problema

- Função que calcula salário de funcionários:
  - Passos são semelhantes para func. públicos e CLT
  - Porém, existem alguns detalhes diferentes
- Na classe pai (Funcionario) queremos definir o "workflow principal" (ou o template) para cálculo de salários
- E deixar aberto para as subclasses os refinamentos desses passos

Solução: Padrão **Template Method**

- Especifica como implementar o esqueleto de um algoritmo em uma classe abstrata X
- Mas deixando pendente alguns passos (ou métodos abstratos) para serem implementados nas subclasses
- Permite que subclasses customizem um algoritmo, mas sem mudar sua estrutura

```java
abstract class Funcionario {

  double salario;
  ...
  abstract double calcDescontosPrevidencia();
  abstract double calcDescontosPlanoSaude();
  abstract double calcOutrosDescontos();

  public double calcSalarioLiquido() { // template method
    double prev = calcDescontosPrevidencia();
    double saude = calcDescontosPlanoSaude();
    double outros = calcOutrosDescontos();
    return salario - prev - saude - outros;
  }
}
```

Implementados pelas subclasses

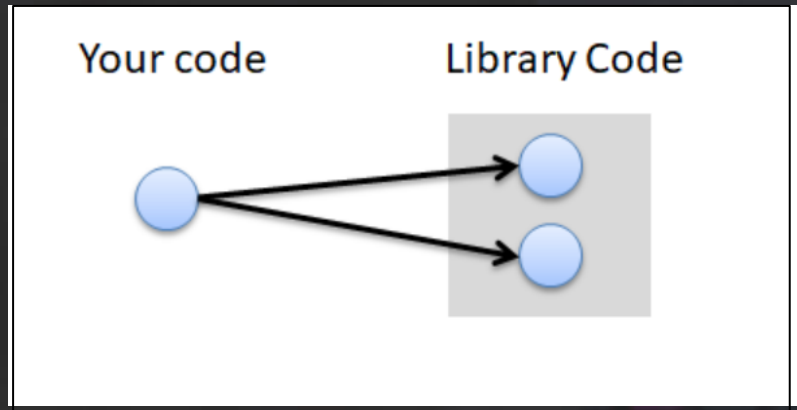
```java
abstract class Funcionario {

  double salario;
  ...
  abstract double calcDescontosPrevidencia();
  abstract double calcDescontosPlanoSaude();
  abstract double calcOutrosDescontos();

  public double calcSalarioLiquido() { // template method
    double prev = calcDescontosPrevidencia();
    double saude = calcDescontosPlanoSaude();
    double outros = calcOutrosDescontos();
    return salario - prev - saude - outros;
  }
}
```

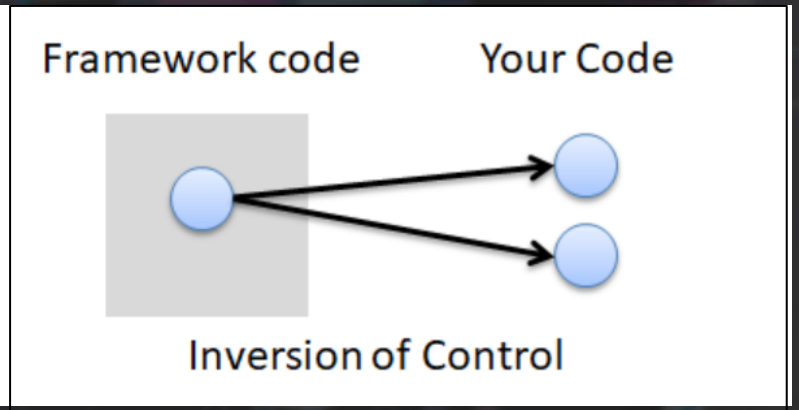
Passos principais (ou template, ou modelo) para cálculo de salário líquido

Inversão de Controle

- Template Method é usado para implementar **Inversão de Controle**, principalmente em frameworks
- Framework: define o "modelo" de um algoritmo/sistema
  - Mas clientes podem parametrizar alguns passos
  - Framework chama esse código dos clientes
  - Daí o termo inversão de controle

Frameworks vs Bibliotecas

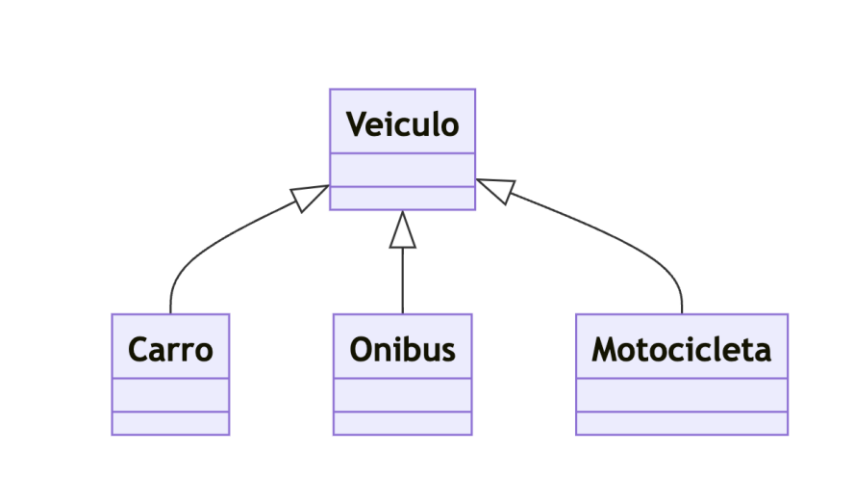




Fonte: https://github.com/prmr/SoftwareDesign/blob/master/modules/Module-06.md

(10) Visitor

Contexto: `Veiculo` e subclasses



E uma lista polimórfica de Veiculos

```
List<Veiculo> list = new ArrayList<Veiculo>();
list.add(new Carro(..));
list.add(new Onibus(..));
...
```

Problema

- Temos que realizar certas operações com os veículos da lista
- Exemplo:
  - Imprimir dados de veículos (sejam `Carro`, `Onibus`,...)
  - Salvar dados de veículos em disco
  - Enviar uma msg para os donos dos veículos

Problema

- Queremos seguir o princípio Aberto/Fechado:
  - Manter `Veiculo` e suas subclasses fechados para mudanças, mas aberto para extensões
  - Extensões: operações que queremos realizar com veículos

Possível alternativa

```
interface Visitor {
    void visit(Carro c);
    void visit(Onibus o);
    void visit(Motocicleta m);
}

class PrintVisitor implements Visitor {
    public void visit(Carro c) { "imprime dados de carro" }
    public void visit(Onibus o) { "imprime dados de onibus" }
    public void visit(Motocicleta m) {"imprime dados de moto"}
}
```

Interface foi criada porque amanhã poderemos ter outros tipos de Visitors (exemplo: que salva os dados em um arquivo JSON)

FURB
UNIVERSIDADE DE BLUMENAU

Possível alternativa (cont.)

```
PrintVisitor visitor = new PrintVisitor();
foreach (Veiculo veiculo: listaDeVeiculosEstacionados) {
   visitor.visit(veiculo); // erro de compilação
}
```

Possível alternativa (cont.)

```
PrintVisitor visitor = new PrintVisitor();
foreach (Veiculo veiculo: listaDeVeiculosEstacionados) {
    visitor.visit(veiculo); // erro de compilação
}
```

- Em Java e linguagens similares, compilador não conhece o tipo dinâmico do parâmetro `veiculo`
- Logo, ele não sabe identificar qual dos três métodos `visit` de `PrintVisitor` deverá ser chamado

Decisão do método a ser chamado é "bi-dimensional"

	PrintVisitor	MailVisitor	SaveVisitor	etc
Carro				
Onibus				
Motocicleta				
etc				

Solução: **Padrão Visitor**

- Permite adicionar uma operação genérica em uma família de classes, sem mexer no código delas
- Simula multiple dispatching, em linguagens que oferecem apenas single dispatching

```
abstract class Veiculo {
  abstract public void accept(Visitor v);
}

class Carro extends Veiculo {
  ...
  public void accept(Visitor v) {
   v.visit(this);
  }
  ...
}

class Onibus extends Veiculo {
  ...
  public void accept(Visitor v) {
    v.visit(this);
  }
  ...
}

// Idem para Motocicleta
```

```
PrintVisitor visitor = new PrintVisitor();
foreach (Veiculo veiculo: listaDeVeiculosEstacionados) {
  veiculo.accept(visitor);
}
```

Chama accept do tipo dinâmico de `veiculo`. Suponha que seja `Carro`

```
class Carro extends Veiculo {
  ...
  public void accept(Visitor v) {
   v.visit(this);
  }
  ...
}
```

Tipo de `this` é conhecido estaticamente (`Carro`)

A escolha do método ocorre em dois passos dinâmicos, mais um passo estático

veiculo.accept(visitor) ⇒

se "veiculo"
Carro
Onibus
Motocicleta
etc

⇒ Carro.accept(visitor) { visitor.visit(this); }

⇒ Onibus.accept(visitor) { visitor.visit(this); }

⇒ Motocicleta.accept(visitor) { visitor.visit(this); }

Decisão apenas em tempo de execução

Decisão em tempo de compilação. Tipo de "this" é conhecido estaticamente

visitor.visit(this) ⇒

se "visitor"	se "this"		
	Carro	Onibus	Motocicleta
PrintVisitor	visit(Carro)	visit(Onibus)	visit(Motocicleta)
etc			

Visitor: vantagens & desvantagens

Visitor: Vantagens

- Visitors facilitam a adição de um método em uma hierarquia de classes
- Pode existir um segundo Visitor, com outras operações
  - Exemplo: calcular imposto de veículos de MG

Visitor: Desvantagens

- A adição de uma nova classe na hierarquia (exemplo: `Caminhao`), implica que todos os Visitors terão que ser atualizados, com um novo método: `visit(Caminhao)`

Visitor: Desvantagens

- Visitors podem quebrar encapsulamento
  - Por exemplo, `Veiculo` pode ter que implementar métodos públicos expondo seu estado interno para que os Visitors tenham acesso a ele

Quando não vale a pena usar padrões de projeto?

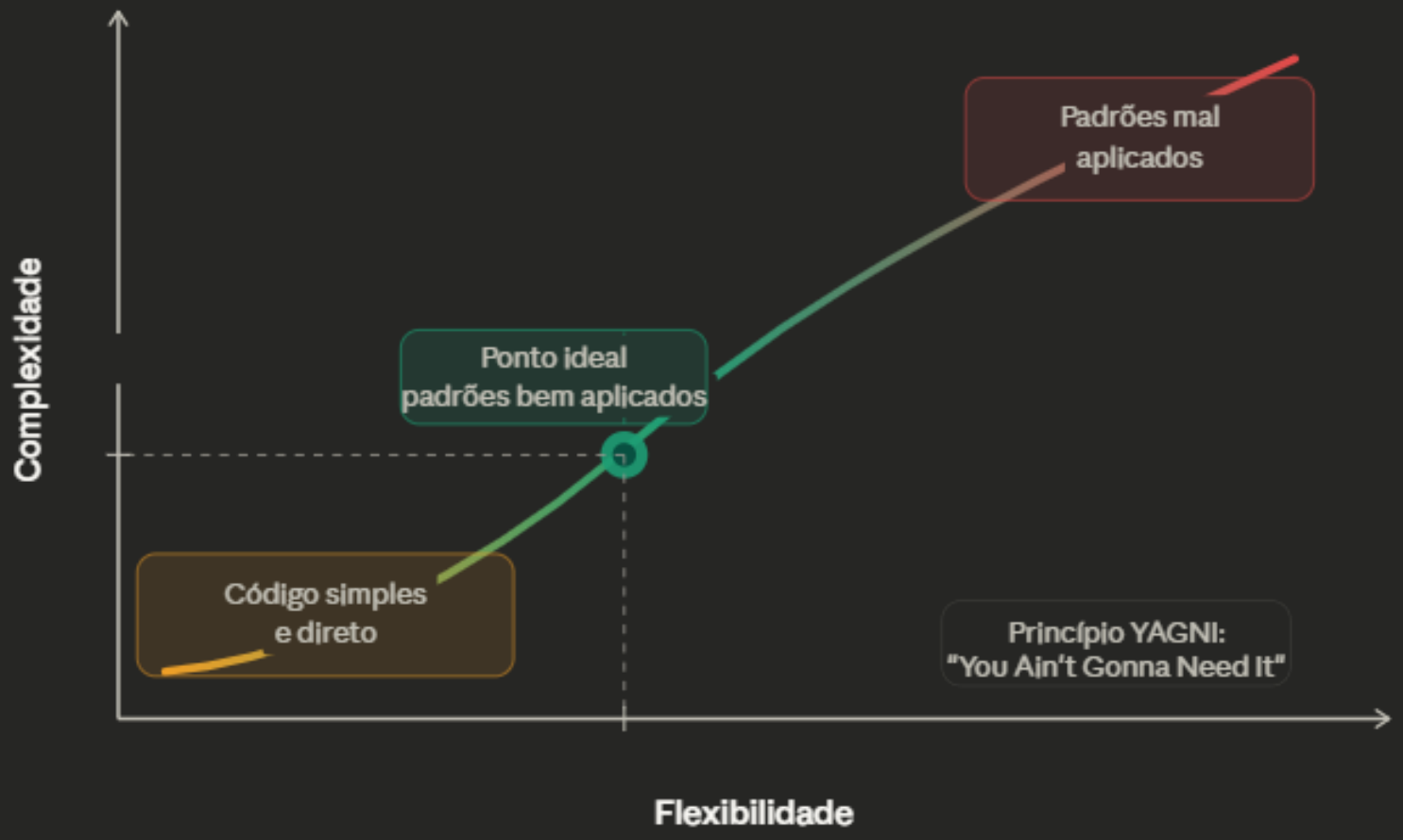
FURB
UNIVERSIDADE DE BLUMENAU

Padrões de Projeto ⇔ Design for change

- Se a chance de mudanças é zero, não precisamos de padrões de projeto

Trade-off: Design for Change vs Complexidade

- Para viabilizar "design for change", Design Patterns complicam um pouco o projeto
- Por exemplo, demandam a criação de classes extras


Complexidade
Flexibilidade
Padrões mal aplicados
Ponto ideal padrões bem aplicados
Código simples e direto
Princípio YAGNI: "You Ain't Gonna Need It"

Exemplo: Strategy

Solução sem padrão de projeto

```
class MyList {

  ... // dados de uma lista
  ... // métodos de uma lista: add, delete, search

  public void sort() {
    ... // ordena a lista usando Quicksort
  }

}
```

1 classe

Solução sem padrão de projeto

```java
class MyList {

  ... // dados de uma lista
  ... // métodos de uma lista: add, delete, search

  public void sort() {
    ... // ordena a lista usando Quicksort
  }

}
```

1 classe

Solução usando Strategy

```java
class MyList {

  ... // dados de uma lista
  ... // métodos de uma lista: add, delete, search

  private SortStrategy strategy;

  public MyList() {
    strategy = new QuickSortStrategy();
  }

  public void setSortStrategy(SortStrategy strategy) {
    this.strategy = strategy;
  }

  public void sort() {
    strategy.sort(this);
  }

}
```

```java
abstract class SortStrategy {
  abstract void sort(MyList list);
}
class QuickSortStrategy extends SortStrategy {
  void sort(MyList list) { ... }
}
class ShellSortStrategy extends SortStrategy {
  void sort(MyList list) { ... }
}
```

1 classe (com mais código)
+1 classe abstrata
+2 classes de estratégias

Mais um exemplo

org.springframework.aop.framework

Class AbstractSingletonProxyFactoryBean

```
java.lang.Object
  └org.springframework.aop.framework.ProxyConfig
      └org.springframework.aop.framework.AbstractSingletonProxyFactoryBean
```

All Implemented Interfaces:

Serializable, BeanClassLoaderAware, FactoryBean, InitializingBean

Direct Known Subclasses:

TransactionProxyFactoryBean

```
public abstract class AbstractSingletonProxyFactoryBean
extends ProxyConfig
implements FactoryBean, BeanClassLoaderAware, InitializingBean
```

Convenient proxy factory bean superclass for proxy factory beans that create only singletons.

Manages pre- and post-interceptors (references, rather than interceptor names, as in ProxyFactoryBean) and provides consistent interface management.

Código no GitHub

https://github.com/afburger/FURB-2026_01-PS_II-115_02_001/tree/main/Padroes_Projeto

Referência
https://refactoring.guru/design-patterns/catalog